

BNN_FCC Contest Submission Report

April 17, 2026

Executive Summary

Submission status	Ready for evaluation. Standard contest verification passed, timing/area were collected with Openflex, and a supplemental coverage testbench is provided for evaluator coverage scripts.
Targeted tradeoff / use case	Maximize fmax for the contest SFC topology. The design targets a frequency-first streaming classifier for $784 \rightarrow 256 \rightarrow 256 \rightarrow 10$, accepting modest on-chip memory use to keep a short, timing-friendly pipeline while preserving AXI4-Stream protocol robustness.
Required contest verification	PASSED: all 50/50 tests completed successfully in <code>verification/bnn_fcc_tb.sv</code> under the contest stress settings <code>TOGGLE_DATA_OUT_READY=1</code> , <code>CONFIG_VALID_PROBABILITY=0.8</code> , and <code>DATA_IN_VALID_PROBABILITY=0.8</code> .
Coverage testbench for evaluator scripts	Use <code>verification/bnn_fcc_coverage_tb.sv</code> with <code>verification/cov_check.sh</code> . This supplemental testbench is separate from the standard contest testbench and is the one intended for Apple-style coverage evaluation.
Max frequency / clock period	622.278 MHz (1.607 ns).
Key resource use	LUT=12,480, REG=11,546, BRAM=16, URAM=14, DSP=0.
Measured stressed-run performance	Average latency=1208.1 cycles/image; average throughput=804.2 cycles/output; average throughput at the 100 MHz verification clock=124344.1 outputs/s.
Supplemental coverage result	55/55 coverage tests passed, 100.00% total covergroup coverage across 7/7 covergroup types, 89/89 instance bins hit, and zero assertion failures.

Important interpretation. The verification testbench intentionally injects input gaps and output backpressure, so the measured throughput is a stressed end-to-end number rather than a peak-throughput number. I therefore report both the raw measured values and simple estimates obtained by scaling the measured cycle counts by the achieved fmax.

Design Goal and Architecture Rationale

This submission is optimized for a system-level use case where timing closure and clock headroom are more important than minimizing every last LUT. The intended deployment model is a streaming image-classification accelerator that is configured once, then processes a long sequence of images while tolerating realistic AXI4-Stream stalls.

- **Top-level dataflow.** The top module `rtl/bnn_fcc.sv` accepts AXI4-Stream configuration data, binarizes 8-bit input pixels on the data input stream, stores intermediate layer activations, and performs a final argmax scan over the output-layer population counts before driving the output stream.
- **Layered compute pipeline.** Each neural-network layer is split into a `layer_memory` block

and a `compute_layer` block. `layer_memory` organizes weights and thresholds for burst-like access by parallel neuron lanes, while `compute_layer` sequences preload / compute / finish phases across those lanes.

- **Parallelism choice.** The timing wrapper `openflex/rtl/bnn_fcc_timing.sv` is configured with `PARALLEL_INPUTS=8` and `PARALLEL_NEURONS={8,8,10}`. For the contest topology, this means the two 256-neuron hidden layers are processed in batches of 8 neurons, while the 10-neuron output layer is processed in a single 10-neuron batch.
- **Why this fits the objective.** The design keeps configuration parsing, memory access, neuron accumulation, and argmax handling in clearly staged blocks. That organization uses more dedicated memory than a fully serialized implementation, but it shortens the critical work per cycle and aligns with the stated goal of maximizing clock frequency.

Timing, Area, and Measured Performance

Timing and utilization were collected with Openflex/Vivado using out-of-context synthesis and implementation of `bnn_fcc_timing`, targeting XCU250-FIGD2104-2L-E.

Metric	Used	Total	Utilization
fmax	622.278 MHz (1.607 ns period)		
LUT	12,480	1,728,000	0.722%
REG	11,546	3,456,000	0.334%
BRAM	16	2,688	0.595%
URAM	14	1,280	1.094%
DSP	0	12,288	0.000%

The standard contest verification flow used `verification/bnn_fcc_tb.sv` with a 100 MHz test-bench clock and the stress settings above. The run completed successfully and reported the following measured performance:

Metric	Value
Average latency (cycles per image)	1208.1
Average latency at 100 MHz	12081.0 ns
Average throughput (cycles per output)	804.2
Average throughput at 100 MHz	124344.1 outputs/s
Estimated throughput at achieved fmax	773784.6 outputs/s
Estimated latency at achieved fmax	1.941 μ s/image

Standard Contest Verification

The required contest verification result comes from the checked-in Openflex/Questa run driven by `openflex/verify.sh`.

- **Testbench and workload.** `verification/bnn_fcc_tb.sv` with the trained MNIST contest topology $784 \rightarrow 256 \rightarrow 256 \rightarrow 10$.

- **Tool provenance.** QuestaSim-64 2023.3 (qrun/vlog/vopt/vsim).
- **Outcome.** PASSED, with all 50 tests completed successfully and no simulation errors reported in the run log.
- **Interpretation.** This is the “required tests” result for submission grading. The coverage testbench discussed below is supplemental and should be treated as additional verification evidence rather than a replacement for the standard testbench.

Unit-Level Verification and Edge Cases

The repository includes a suite of self-checking unit testbenches in `verification/` that target individual RTL modules and boundary conditions without requiring the full end-to-end contest testbench. The unit suite is run by `verification/run_unit_tests.sh`, which compiles each bench into an isolated work area under `verification/.unit_sim/`, runs it in batch mode (`vlog/vsim`), and requires an explicit `SUCCESS:` banner from each bench.

- **config_parser_unit_tb (64-bit) and config_parser_unit_tb_32 (32-bit).**
Validate `config_parser` header/payload sequencing across different bus widths. Checks include `config_ready` reset behavior, correct header decoding, propagation of `TKEEP` into write strobes, and sequential write addresses across payload beats. Directed edge cases include idle gaps between beats (64-bit bench), auto-termination when `total_bytes` fits in one beat even without `TLAST`, early `TLAST` termination, suppression of writes for unsupported `msg_type` values or invalid `layer_id` values, and the `layer_id` $\rightarrow$ write-enable mapping (`bit = layer_id+1`).
- **layer_memory_unit_tb.**
Exercises `layer_memory` writes plus batched reads, checking the expected pipelined read latency, weight alignment (including byte-offset slices that straddle 64-bit word boundaries), threshold packing, partial threshold updates via byte strobes, and that threshold outputs clear while reset is asserted.
- **layer_memory_edge_unit_tb.**
Forces a tiny configuration that collapses the weight memory to `WEIGHT_MEM_DEPTH=1` and packs `BYTES_PER_NEURON=1`. It verifies packed single-byte neuron weights, partial threshold writes to upper vs. lower 32-bit halves via strobes, and that the final batch returns zeros on the out-of-range parallel lane (weights and thresholds).
- **neural_processor_unit_tb.**
Checks XNOR-popcount accumulation and threshold compare for `N=8` across single-chunk and multi-chunk transactions, verifies `valid_out` is a one-cycle pulse, and confirms the accumulator resets between transactions (no carry-over).
- **neural_processor_edge_unit_tb.**
Stresses back-to-back `valid_in` chunks with no idle cycles for `N=4` (minimum supported divisibility) and `N=8`. It also changes `threshold` between chunks and confirms that the threshold presented with the final `last` chunk is the one applied, then immediately starts a new transaction to confirm accumulator reset at the boundary.

- **compute_layer_unit_tb.**
Covers the single-chunk controller path in both hidden-layer and output-layer modes. It verifies `mem_layer_start` assertion on `valid_in`, correct output formatting (binarized bits vs. raw per-neuron popcounts), and that no additional memory read requests are issued in the single-chunk case.
- **compute_layer_multichunk_unit_tb.**
Forces multi-chunk operation by using `LAYER_INPUTS=37` with `CONFIG_BUS_WIDTH=8` (five chunks per neuron) and uses a simple pipelined memory model to mimic `layer_memory` latency. It confirms that `mem_read_weight` pulses during multi-chunk execution in both hidden/output modes, that padded bits in the final chunk do not affect results, that outputs remain X-free, and that hidden-layer backpressure is handled correctly by holding `valid_out` and the result vector stable until `ready_in` is asserted.

Coverage Methodology and Coverage-Plan Mapping

For Apple-style coverage evaluation, use `verification/bnn_fcc_coverage_tb.sv`.

Run it with `verification/cov_check.sh`. The bench switches to the smaller custom topology $13 \rightarrow 10 \rightarrow 10 \rightarrow 5$, which intentionally forces partial final beats on the 64-bit input stream so that TKEEP handling is exercised in a controlled way. The run completed 55/55 checks successfully and reached 100.00% total covergroup coverage across all 7 covergroup types.

Coverage plan item	How it is addressed in <code>verification/bnn_fcc_coverage_tb.sv</code>	Evidence
Category 1: AXI4-Stream protocol patterns	<code>update_probabilities()</code> rotates configuration and image valid patterns between continuous, intermittent, and bursty modes. The output-ready driver rotates between always-ready, intermittent ready, and long backpressure bursts. The 13-input custom topology forces partial input beats, and the sequencer explicitly exercises TLAST boundaries.	Config AXI 9/9; image AXI 10/10; output AXI 8/8.
Category 2: configuration data diversity	<code>create_diverse_config()</code> constructs all-zero, sparse, balanced, dense, and all-one weight patterns, plus small / medium / large threshold classes. Threshold “don’t care” fields are intentionally randomized in part of the configuration stream.	Configuration diversity 31/31.
Category 3: computational stimulus	<code>retune_model_for_cat3()</code> synthesizes examples that hit every output class in the custom 5-class topology. A directed repeated/varying class sequence is prepended, then 50 random vectors are added to broaden data-space exploration.	Classification 7/7.
Category 4: configuration- image sequencing	The sequencer performs an initial partial configuration, a full reconfiguration, weights-only partial reconfiguration, thresholds-only partial reconfiguration, weights+thresholds partial reconfiguration, and an additional full reconfiguration while image traffic is being exercised.	Reconfiguration 12/12.
Category 5: reset scenarios	Resets are injected before normal traffic, during configuration, on an image TLAST boundary, during output draining, and after the output stream has drained. Both same-configuration and different-configuration post-reset cases are covered, along with cold/warm/hot workload buckets.	Reset 12/12.

Name	Class Type	Coverage	Goal	% of Goal	Status	Included	Merge_instances	Get_inst_coverage	Comment
/bnn_fcc_cat1_tb		100.00%	100	100.00%					
TYPE_cg_axi_config		100.00%	100	100.00%				auto(0)	
TYPE_cg_axi_image		100.00%	100	100.00%				auto(0)	
TYPE_cg_axi_output		100.00%	100	100.00%				auto(0)	
TYPE_cg_config_diversity		100.00%	100	100.00%				auto(0)	
TYPE_cg_classification		100.00%	100	100.00%				auto(0)	
TYPE_cg_reconfiguration		100.00%	100	100.00%				auto(0)	
TYPE_cg_reset		100.00%	100	100.00%				auto(0)	

Figure 1: Supplemental coverage run summary. All seven coverage-plan covergroup types reached 100% coverage.

Across the seven covergroup instances, all 89/89 bins were hit: 9/9 for configuration AXI behavior, 10/10 for image AXI behavior, 8/8 for output AXI behavior, 31/31 for configuration diversity, 7/7 for class coverage, 12/12 for reconfiguration behavior, and 12/12 for reset behavior. The coverage report also records zero assertion failures.

Code Coverage

The filtered code-coverage summary for the supplemental coverage run is 71.77%. For the main RTL files, the branch-coverage breakdown is:

Module	Branch coverage
rtl/bnn_fcc.sv	86.04%
rtl/compute_layer.sv	92.85%
rtl/config_parser.sv	81.81%
rtl/layer_memory.sv	85.18%
rtl/neural_processor.sv	91.66%

The remaining misses are concentrated in generic or defensive branches that are not central to the contest operating point: elaboration-time parameter guards, fallback paths for unused parameterizations, and generic bounds/default handling in the parameterized parser / memory / argmax logic. In other words, the directed coverage-plan scenarios are fully exercised, while some reusable safety logic remains naturally unhit under the contest topologies and parameter settings.

Design Space Exploration Summary

The file `syn/results.csv` records a sweep of Openflex synthesis and implementation checkpoints across 35 distinct architectural iterations. The report categorizes these sweep points to illustrate the diverse design space explored. Selected checkpoints highlighting different design paradigms are:

Sweep point	fmax	LUT	REG	BRAM	URAM
Peak F_{max} (Final Submission)	622.665	12462	11545	16	14
Balanced Trade-off Baseline	417.711	9149	8146	16	14
Minimum Registers (Lowest REG)	413.394	9085	7685	16	14
Minimum Area (Lowest LUT)	354.359	8848	8093	16	14
Suboptimal Exploration (Worst case)	258.131	16026	24172	16	14

Relative to the balanced baseline configuration, the final submission improves frequency by 204.954

MHz (49.07%). Across the entire design sweep, BRAM and URAM usage remained perfectly constant at 16 and 14 blocks respectively; ensuring the on-chip memory footprint was invariant. Consequently, the primary axis of exploration involved balancing LUT and REG utilization against the maximum achievable clock frequency (F_{max}).

To thoroughly investigate this tradeoff, the highlighted design choices encapsulate distinct hardware allocation strategies:

- **Peak F_{max} (Final Submission):** The final submitted design focuses on maximizing clock headroom. By purposefully relaxing resource constraints, we deepened the pipeline and eliminated logic bottlenecks, achieving an exceptional F_{max} of 622 MHz.
- **Balanced Trade-off Baseline:** An intermediate checkpoint that reasonably balanced logic scale and achievable speed. It served as a stable baseline with very competitive resource densities.
- **Minimum Registers (Lowest REG) and Minimum Area (Lowest LUT):** These configurations aggressively constrained hardware area by restricting pipeline depth. While they achieved the absolute lowest element counts, their critical paths necessarily lengthened, yielding lower clock frequencies.
- **Suboptimal Exploration (Worst case):** Included to demonstrate wide exploration, this experimental checkpoint attempted to aggressively split computational data paths. The resulting fan-out bloat and routing congestion caused resource utilization to skyrocket and F_{max} to plunge, perfectly illustrating the limits of structural over-sharding.

Ultimately, this comprehensive exploration verified that the final architecture optimally provisions FPGA fabric to satisfy the contest’s primary objective: peak operating frequency for streaming classification.

Reproducibility

Standard contest verification

```
cd openflex
./verify.sh
```

Timing and area collection

```
cd openflex
openflex bnn_fcc_timing.yml -c bnn_fcc.csv
```

Supplemental coverage run

```
cd verification
./cov_check.sh
```

Module-level unit tests

```
cd verification
./run_unit_tests.sh
```

Evaluator note. If your coverage scripts require an explicit top-level testbench, point them at `verification/bnn_fcc_coverage_tb.sv` and use `verification/cov_check.sh`. The standard contest result should still be evaluated with `verification/bnn_fcc_tb.sv`.

Tool Versions

- Vivado v2021.2
- QuestaSim-64 2023.3 (qrun/vlog/vopt/vsim)